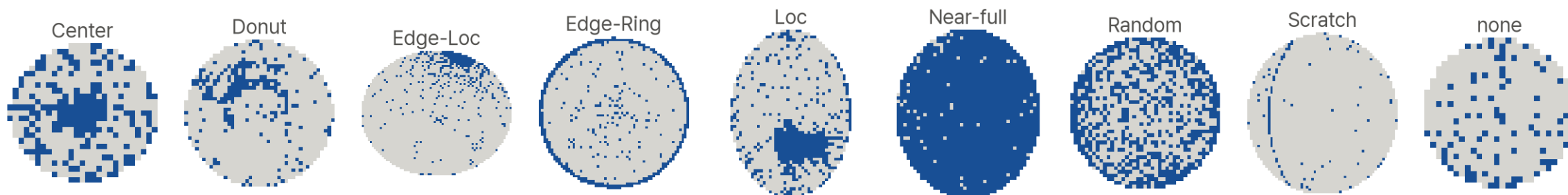


웨이퍼 맵 불량 패턴 분류

lot 단위로 나눈 평가에서 macro-F1 0.909를 확인한 소형 CNN

WM-811K 공개 데이터 | 개인 프로젝트 | 2026. 9.



이진성

서울과학기술대학교 전기정보공학과 석사과정

학습에 쓰지 않은 lot에서 macro-F1 0.909를 확인했습니다. 평가 방식을 먼저 고치고, 결과를 다섯 가지 방법으로 검증했습니다.

기간·형태	2026년 9월, 개인 프로젝트(1인)
데이터	WM-811K 웨이퍼 맵 811,457장 중 라벨 있는 172,950장, 불량 패턴 9종
문제	같은 웨이퍼의 복사본이 학습과 평가에 섞이는 누수를 막고, 새 lot에서의 성능을 재기
결과	macro-F1 0.843(베이스라인) → 0.909(CNN), 모든 패턴 recall 0.77 이상
역할	문제 정의·평가 설계·판단·검증: 본인 / 코드 구현·실험·문헌 조사: Claude Code(AI 코딩 도구)
환경	Python 3.12, PyTorch 2.7(CUDA 11.8), scikit-learn 1.9.1, Streamlit, Quadro P2000

lot 같은 공정 조건으로 함께 처리한 웨이퍼 묶음(최대 25장). 한 lot의 맵은 서로 닮아서 분할의 기본 단위로 씀
macro-F1 패턴 9종 F1의 단순 평균. 85%를 차지하는 none에 점수가 치우치지 않음
recall 실제 그 패턴인 웨이퍼 중 맞힌 비율

진행 순서

- 1 데이터 탐색과 누수 확인 3쪽
- 2 평가 설계: lot 단위 분할, 성공 기준 3쪽
- 3 베이스라인과 소형 CNN 4쪽
- 4 검증 다섯 가지와 오류 수정 5쪽
- 5 AI 협업 방식 6쪽
- 6 결론·한계·재현 정보 7쪽

02 문제 정의: 데이터와 평가 방식

원본 분할과 무작위 분할은 같은 웨이퍼의 복사본이 학습과 평가 양쪽에 들어가 점수가 부풀려집니다. 그래서 lot과 복사본을 한 묶음으로 나눴습니다.

누수 3겹

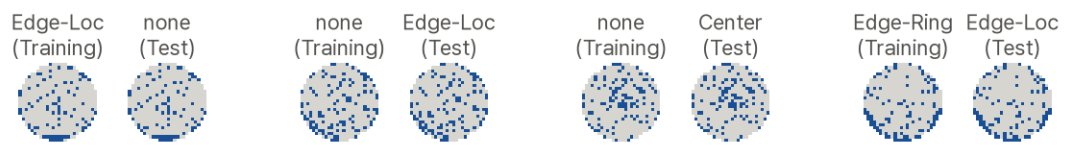
원본 분할	test 3,158장이 train의 복사본. 같은 맵에 다른 라벨이 붙은 쌍 14개
lot 단위 분할	재검사로 생긴 거의 같은 맵이 양쪽에 남음(Center 패턴은 40.7%)
반전·회전 복사본	뒤집거나 돌린 복사본도 있어 8가지 변환을 모두 비교해서 묶음

데이터 라벨 있는 172,950장 사용. none 85.2%, Near-full 149장(0.09%)으로 불균형

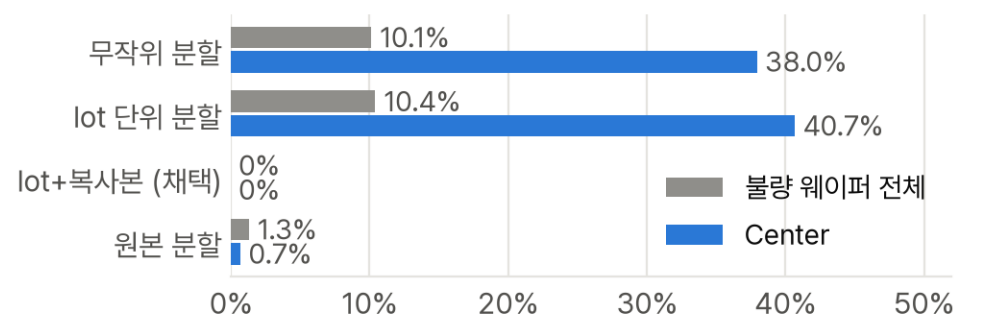
채택한 분할 lot + 10픽셀 이내 복사본(반전·회전 포함)을 한 묶음으로 70 / 15 / 15

성공 기준 test macro-F1 0.80 이상, 모든 패턴 recall 0.50 이상, 베이스라인 초과. 신뢰구간 병기

같은 맵, 다른 라벨: 원본 Training/Test 사이 복사본 14쌍 중 4쌍



test 웨이퍼 중 학습 쪽에 10픽셀 이내 복사본이 있는 비율



같은 데이터·같은 분할·같은 test 25,444장에서 소형 CNN이 베이스라인보다 macro-F1 0.066 높았고, Loc·Scratch recall이 가장 크게 올랐습니다.

모델	macro-F1 (95% CI)	최저 recall
베이스라인: 특징 23개 + 랜덤 포레스트	0.843 (0.820–0.858)	0.497 (Loc)
소형 CNN, CE	0.901 (0.863–0.918)	0.763 (Loc)
소형 CNN, CE + val 보정 (최종)	0.909 (0.885–0.922)	0.775 (Loc)

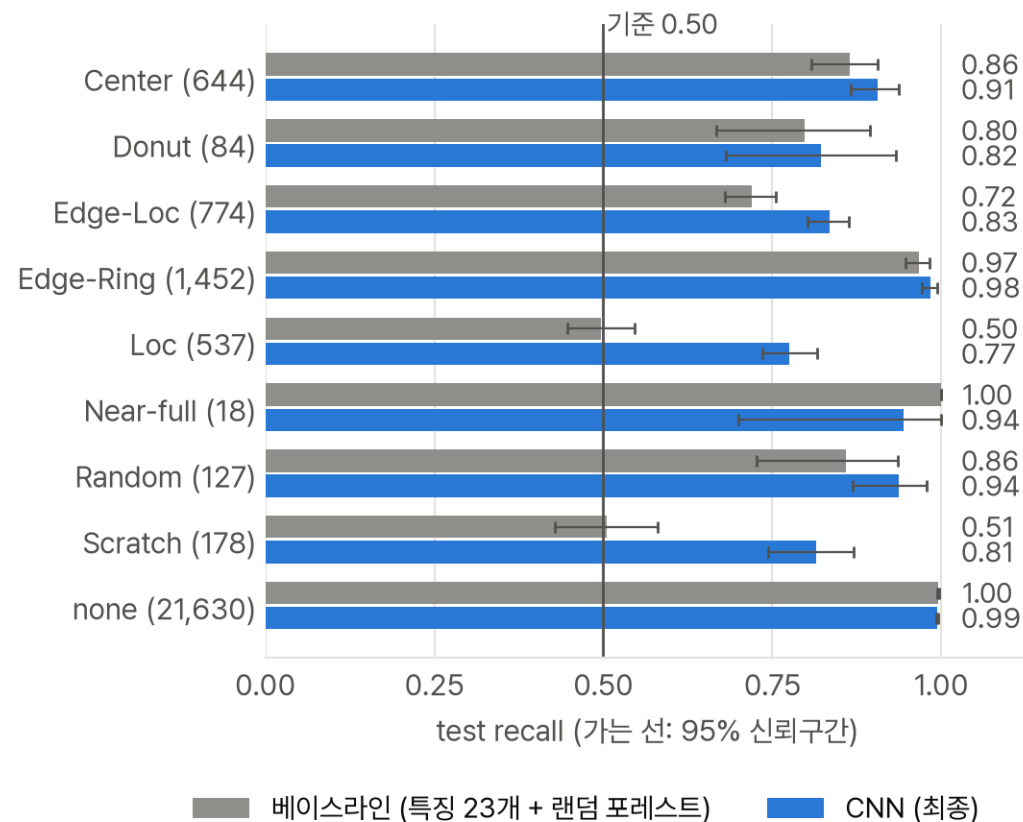
입력 64×64로 축소. 축소할 때 블록 최대값을 써서 1다이 폭 Scratch를 남김.

반전·회전 8종 증강

모델 합성곱 블록 4개, 파라미터 58만, CE 20 epoch(GPU 40분). 패턴별 점수 보정값은 val에서만 정함

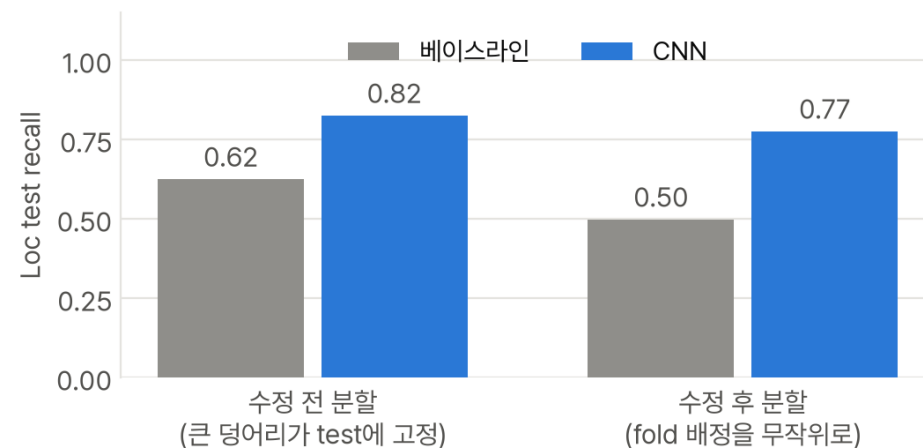
효과 없던 시도 class-weighted CE: macro-F1 0.825, none 오탐 증가 → 미채택

용도 엔지니어가 어떤 웨이퍼를 먼저 볼지 정하는 1차 분류 보조. 자동 폐기 판정에는 부적합



점수가 예상보다 높게 나온 이유를 확인하려고 다섯 가지를 검증했고, 그 과정에서 분할 절차의 오류 하나를 찾아 고쳤습니다.

검증	방법	결과
우연 수준	라벨을 섞어 같은 CNN 학습	macro-F1 0.102 (우연 수준)
누수 제거 효과	무작위 분할 vs lot 분할, 같은 모델	0.895 vs 0.909, 차이 없음
지름길 학습	25×27 제품 외곽선을 다른 제품 것으로 교체	예측 99.5% 이상 유지
오분류 검토	96장을 라벨 가린 채 독립 판정 2회	애매 46% / 모델 오류 27% / 라벨 의심 27%
재현	새 폴더에 clone 후 README 순서로 재실행	1시간 8분, 비교 항목 10개 일치



분할 절차 오류 (LOG E11)

원인 scikit-learn 1.9.1 StratifiedGroupKFold가 크고 치우친 그룹을 seed와 무관하게 같은 fold에 배정. 맞추기 쉬운 Loc 웨이퍼 132장(28개 lot)이 항상 test에 들어감

발견 분할 규칙을 바꾼 뒤 베이스라인 점수가 이유 없이 오름 → seed 10개로 다시 나눔(Loc recall 최대 15%p 차이) → lot 묶음 하나만 test에서 빼 보는 실험

수정 fold와 분할의 대응을 seed로 무작위 치환하고 모든 모델 재학습(0.893 → 0.909). 분할 직후 train-test 겹침과 lot 묶음 비중을 검사하는 단계 추가

05 AI 협업 방식

문제 정의와 기준, 최종 판단은 제가 내리고 구현과 실험은 Claude Code에 맡겼습니다. 결과는 제가 정한 절차로 검증했습니다.

본인	문제 정의, 정제·분할 규칙, 성공 기준, 모델 채택·폐기, 최종 수치 채택, 오류 판정 (판단 13건)
Claude Code	코드 구현, 실험 실행, 문헌·도구 조사, 독립 에이전트의 반박 검증 (오류 15건 중 7건 발견)
작업 규칙	단계마다 멈춰 선택지 2~3개와 근거 제시 / 수치는 실행 결과만, 스크립트 명시 / 오류는 발견자·경위·조치까지 기록

지시 원문

“내가 100프로 만족할 수 있게 나에게 질문하면서 진행해” — 첫 지시.
 체크포인트마다 멈추게 한 근거

“대시보드의 문체를 다른 사이트를 참고해서 바꿔줘 너무 ai틱해” — 산출물 반려.
 참고 사례를 조사한 뒤 문구를 다시 씀

기록 지시 원문·판단·오류 → LOG.md / 계획과 변경 이력 → PLAN.md / 수치·버린 것·한계 → RESULT.md

근거: LOG.md 판단 지점 1~13, 오류 기록 E1~E14, docs/wafer_project_prompt.md

판단	선택	근거
라벨 없는 638,507장	제외	불량 0개 맵이 5.8만 장 등 분포가 다르고, 평가 맵의 복사본 5,408장이 섞여 있음
성공 기준	macro-F1 0.80 + 전 패턴 recall 0.50	선행연구의 lot 분할 0.85는 재검사 복사본 누수 가능성이 크고, 같은 맵의 라벨이 갈리는 노이즈가 있음
불균형 처리	가중 손실 대신 val 사후 보정	같은 test에서 비교: weighted CE 0.825 < CE 0.901
분할 오류 대응	절차 수정 후 전부 재학습	얕은 맵 묶음 규칙은 유지하고 fold 배정만 무작위로 바꿈. 이전 결과는 '편향된 분할의 결과'로 표시해 남김

이 결과는 같은 제품의 새 lot에 대한 1차 분류 보조 수준입니다. 수율 개선이나 공정 원인 규명 효과는 주장하지 않습니다.

결론

- 누수를 막은 평가에서 CNN 0.909, 베이스라인 0.843
- 분할 절차 하나로 Loc recall이 12%p 달라짐. 모델을 고르기 전에 평가 방식부터 확인해야 함
- 검증 5종 통과, 새 폴더 재현 일치

한계·다음 단계

- 한 번 나눈 분할의 결과. seed 10개로 나눠 평균하면 약 0.01 낮음 → 그룹 5-fold로 전체 평가
- Near-full test 18장, 신뢰구간 0.70~1.00 → 표본 확보 전까지 수치 유보
- 같은 맵 다른 라벨 14쌍, 6~10다이 짧은 Scratch 약점 → 라벨 기준 재정의, 고해상도 입력

배운 점

- 수치는 스크립트 실행 결과에서만 옮긴다. 추정치는 쓰지 않는다
- 점수가 갑자기 오르면 원인을 찾기 전까지 채택하지 않는다
- [본인 문장 한 줄]

재현 정보

환경 Windows 11, Python 3.12.6, torch 2.7.1+cu118, scikit-learn 1.9.1, Quadro P2000. 시드 0 고정

실행 python scripts/run_all.py (데이터 다운로드 → 분할 → 베이스라인 → CNN → 평가, 약 4~5시간). 결과 outputs/*.json

저장소 github.com/sl2ver/wm811k-wafer-defect-classification (MIT) · PLAN.md 계획·변경 이력 / LOG.md 지시 원문·판단 13건·오류 15건 / RESULT.md 수치·버린 것 9·한계 9